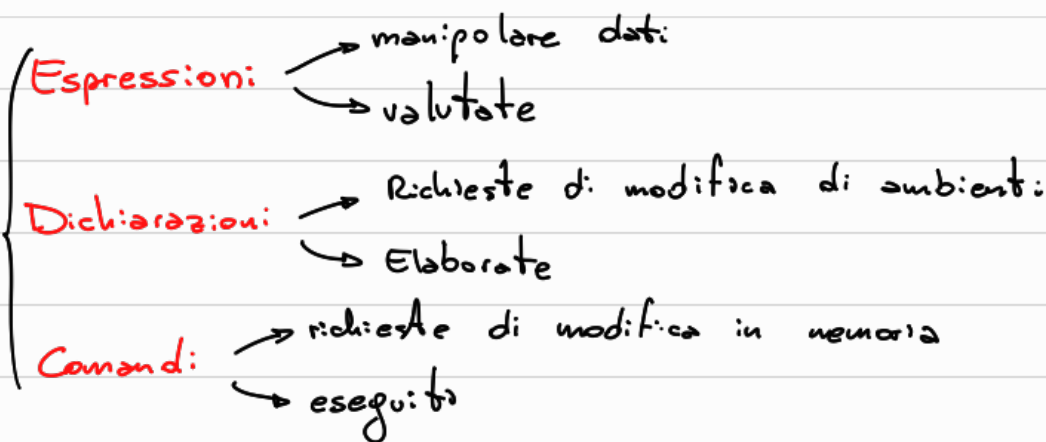




Espressioni (denotano dati)

$$2+4 \neq 1+5$$

$$L \equiv \uparrow$$

6

Valori esprimibili $\Rightarrow$ valori esprimibili (denotabili) attraverso la sintassi:

de il PL mette a disposizione per le espressioni:

$$\text{Eval} = \text{Bool} \cup \text{int}$$

$$\{\text{tt}, \text{ff}\}$$

$$\mathbb{Z}$$

$\Rightarrow$ **Espressioni**: aritmetiche : aspetti progettuali

$\hookrightarrow$ **Arietà delle operazioni**: (notazione)

Arietà = # operatori di un'operazione

infissa $\swarrow$ pre-fissa $\downarrow$ postfissa $\searrow$

$\hookrightarrow$ Regole di associatività

$\hookrightarrow$ Regole di precedenza

$\hookrightarrow$ Ordine di valutazione degli operandi

$\hookrightarrow$ Presenza di side-effects

$\hookrightarrow$ Overloading di operatori

$\hookrightarrow$ Espressioni con operandi di tipo misto (it+ciaa)

Notazione

- infissa $a+b$
- pre-fissa $+ab$
- postfissa $ab+$

} non richiedono regole associatività e precedenza $(a-b)-c \neq a-(b-c)$

postfissa 532*+ 2 2 arieta
 ↑ ↑



pre-fissa +*325 arieta che memorizza il numero di operandi dell'operatore da applicare



infixa $a+b*c$ $**d**e/f$ le regole di precedenza servono per descrivere quale operatore applicare prima
 $2+5*2$

Regole di associatività decidono l'ordine di applicazione di operatori allo stesso livello di precedenza

L'ordine di VALUTAZIONE degli operandi

[0:37:00]

operandi definiti 1 non side-effect 2 aritmetica finita 3

1) $a == 0 ? b : b/a$ se $a == 0$ allora b/a altr. b

→ valutazione eager → valuta tutto

→ valutazione lazy → valuta solo quello che serve

$p := lista;$

while ($p \neq nil$) and ($p.value \neq 3$) do { ... }

$p = nil$

$p.value \Rightarrow$ errore x

} valutazione eager

$p = \text{nil}$ } val. lazy $\Rightarrow$ ff ✓

2) side-effects

$a := 10;$

$b := a + \text{fun}(\&a);$

fun modifica il suo parametro

$a := 12$

prendo $a = 10$ o faccio prima $\text{fun}(\&a)$?

$10 + 12$

$12 + 12$

[0:45:00]

3) aritmetica finita

Espressioni IMP

booleana

$\langle \text{Exp} \rangle : E \rightarrow A | B$

aritmetica

con reg. di associatività e prec.

$A \rightarrow \overset{\text{terminale}}{n \in \mathbb{Z}} | A \text{ op } A \quad \text{op} = \{+, -, *\}$

$B \rightarrow \text{true} | \text{false} | A = A | \text{not } B | B \alpha B$

N ma insieme dei valori numerici n, m, p

E ma insieme di alberi di valutazione di espressioni e (stringa di soli terminali generati in exp)

Diamo semantica definendo un sistema di transizione

Stati: $E \cup N$ (stati terminali)

Relazione Transizione: definita attraverso regole costruite induttivamente sulla grammatica

$\text{op} \in \{+, -, *\}$

simbolo

$E_1: m \text{ op } n \rightarrow p$ dove $p = \underbrace{m \text{ op } n}_{\text{calcolo effettivo}}$

n, m simboli N

$10 + 12 \rightarrow 22$

$E_2: \frac{e_0 \rightarrow e_0'}{e_0 \text{ op } e_1 \rightarrow e_0' \text{ op } e_1}$

$E_3: \frac{e_1 \rightarrow e_1'}{m \text{ op } e_1 \rightarrow m \text{ op } e_1'}$

[1:02:00]

E_3 .

$$\frac{(2+3)*}{e_0} \frac{(5-(1+4))}{e_1}$$

